

AI駆動開発ガイド

小さく任せ、実行して確かめる。
仕様からレビューまでの実践ガイド

01 定義する

目的・制約・受け入れ条件

02 つくる

調査・実装・テスト

03 確かめる

証拠・レビュー・改善

このガイドで、最初の1件を完了できる

全体像をつかみ、テンプレートを使い、確認項目に沿って実践する。

01	理解する	人とAIの役割、任せる仕事、基本の流れ	03—05
02	依頼する	実行環境、渡す情報、依頼文の型	06—08
03	つくって確かめる	仕様例、実装、テスト、調査、レビュー	09—14
04	チームに定着させる	運用ルール、効果測定、導入計画	15—19

人が目的を決め、AIと検証のループを回す

本資料では、AIを要件整理・設計・実装・検証に組み込む開発の進め方を「AI駆動開発」と呼ぶ。

人	AI	実行環境
目的と判断を持つ 何を解決するか決める 受け入れ条件を定める 差分を理解し採用する	調べて形にする 既存コードを調査する 設計案と変更をつくる テスト案・説明をつくる	結果を返す テストとビルドを実行 静的解析で問題を検出 画面やログで挙動を確認

完了の条件は、期待した挙動を確認でき、変更の責任者が説明できること。

最初は、正誤を確かめやすい仕事を選ぶ

選定の3条件：範囲が小さい・期待結果が明確・元に戻せる。

候補	任せる作業の例	完了を判断する証拠
単体テストの追加	既存関数の境界値を追加	期待値の根拠＋テスト結果
小さな不具合修正	再現条件が明確なバグを修正	修正前に失敗、修正後に成功
説明文の更新	コードとREADMEの差を修正	参照元と手順の実行結果
認証・DB移行	まずは影響調査と案の比較	担当者の設計確認＋移行検証

タスクが大きいときは「調査」「仕様の整理」「1つの変更」に分ける。

1つの変更を、5つの工程で進める

各工程で残す成果物を決めると、会話の途中からでも作業を再開できる。



PR（プルリクエスト）：変更内容を共有し、レビューして取り込むための単位。

使い方に合わせて、実行できる範囲を決める

組織で利用できるツールを選び、編集・コマンド実行・外部接続の範囲を確認する。

補完

書いている箇所の
候補を得る

候補の採用前に
挙動と依存関係を読む

対話

仕様・設計の相談や
コードの説明を得る

回答の参照元と
前提条件を確認する

エージェント

複数ファイルの変更や
テスト実行を任せる

編集範囲・実行権限・
外部接続を制御する

開始前に、作業ブランチ・テスト環境・既存テストの結果をそろえる。

必要な情報を、参照先と優先順位で渡す

判断に使う仕様、対象ファイル、実行方法を短くそろえる。

01

タスク仕様

目的・入出力・受け入れ条件・対象外

02

リポジトリ

関連ファイル・既存の実装例・設計判断

03

実行環境

セットアップ・テストコマンド・再現ログ

参照のルール

最新の仕様を明示する
該当ファイルを絞る
バージョンを伝える
不明点は推測と区別する
決定事項を更新する

長い作業では「決定事項・変更・検証済み・未解決」の4点を引き継ぐ。

依頼文は、目的・範囲・完了条件まで書く

コピーして使うテンプレート例：次ページの `normalize_title` を実装する。

目的	申請メモのタイトルを、保存前に整える。
対象・仕様	<code>src/title.py</code> とテスト。条件は p.09 に従う。
参照・制約	既存の実装に合わせる。公開APIと依存は維持。
進め方	関連箇所を確認し、短い計画を示して実装。
完了条件	対象テストと既存テストを実行して確認。
報告形式	差分の要点／実行コマンドと結果／未検証事項。

「できた」を 判定できる依頼に
何が変わるか
どこまで変えるか
何を実行するか
何を提出するか

受け入れ条件は、入力と期待結果で決める

題材：normalize_title(value) — 申請メモのタイトルを保存前に整える関数。

観点	受け入れ条件	入力 → 期待結果
前後の空白	前後を除去し、内部は保つ	「 申請 メモ 」 → 「申請 メモ」
必須入力	空文字・空白のみはエラー	「 」 → ValueError
最大長	除去後の長さは1～50	50文字 → 成功／51文字 → エラー
型	文字列以外はエラー	123、None → TypeError

数え方も仕様にする：ここでは Python の len() を使う。DB保存・画面表示は対象外。

変更は、レビューできる小ささに保つ

受け入れ条件に必要な差分をつくり、途中で変更範囲を確かめる。

01

既存の実装を読む

同じ責務の処理・例外・テストを確認

02

変更範囲を示す

関数とテストの対象ファイルを列挙

03

1つの仕様を実装する

型 → 空白除去 → 必須 → 最大長の順

04

差分と実行結果を確認

無関係な修正がないかも読む

変更の単位

1つの目的

1つの確認可能な結果

1つの説明できる差分

複雑なら、先に分割する。

テストの期待値は、受け入れ条件から決める

AIが書いた実装とテストが、同じ誤解を共有する可能性を考える。

種類	テスト入力の例	期待する結果
正常系	「申請 メモ」	「申請 メモ」
境界値	「あ」 × 1、50、51	1・50は成功、51は ValueError
異常系	空文字、半角空白のみ	ValueError
型違い	123、None	TypeError

提出する証拠：対象テスト+既存テストのコマンド、結果、未実行の範囲。
修正タスクでは、再現テストが「修正前に失敗 → 修正後に成功」することも確認する。

行き詰まったら、再現条件に戻る

修正を重ねる前に、観測した事実と原因の仮説を分ける。

AIに渡す調査メモ

期待：50文字のタイトルを受け付ける

実際：入力によって ValueError になる

条件：入力値・操作手順・バージョン

証拠：エラー全文・関連ログ・差分

依頼：仮説と確認方法を提示する

01

再現する

最小の入力と手順に絞る

02

仮説を確かめる

1回に1つの原因を検証する

03

回帰を防ぐ

再現テストを残して修正する

説明が変わるだけで証拠が増えないときは、差分を広げず状況を整理する。

レビューでは、差分と証拠を一緒に読む

AIレビューの指摘も検証し、採用する変更は担当者が説明できる状態にする。

レビューの5観点

仕様	受け入れ条件と対象外を守る
構造	既存設計に沿い、差分が適切
安全	権限・入力・機密情報を確認
証拠	テストと解析の実行結果がある
運用	ログ・影響・戻し方を説明できる

PR説明文の型

目的・変更

何が、どのように変わるか

検証結果

実行コマンドと結果、必要なら画面

未検証・影響

確認できていない範囲と利用者への影響

戻し方

切り戻し手順と注意点

データと実行権限を、必要な範囲に絞る

ツールの管理設定と実行環境で制御し、重要な操作には確認を入れる。

入力データ

承認済みの環境を使う

秘密鍵や実データを除く

保持・学習利用の設定を確認

実行権限

作業範囲と接続先を絞る

本番の認証情報を分離する

操作と実行結果を記録する

外部コンテンツ

文書やコメントを資料扱い

混入した命令を採用しない

要求外の外部送信を止める

本番変更・データ削除・外部送信は、対象と影響を示して担当者が判断する。

チームで共有するルールは、短く保つ

人が読む開発ガイドと、AIが参照する指示を、同じ運用ルールにそろえる。

決めること	最低限そろえる内容	主な管理者の例
利用範囲	ツール・データ・外部接続・重要操作	管理者／セキュリティ
開発の約束	ディレクトリ・設計方針・依存の扱い	テックリード
検証の手順	セットアップ・テスト・解析コマンド	開発チーム
完了の基準	必須チェック・レビュー担当・証拠	開発責任者

ルールの管理者と更新タイミングを決め、コードや環境の変更に追従させる。

よくある失敗は、作業の進め方から直す

出力だけを直し続けず、仕様・情報・検証・分割のどこに原因があるかを見る。

起きがちなこと	考えられる原因	次に変える行動
生成のたびに方針が変わる	仕様・判断が未確定	決定事項と対象外を明記
大量の差分を読み切れない	タスクの単位が大きい	1つの目的ごとに分ける
成功の説明だけで完了する	実行可能な確認がない	実行コマンドと結果を求める
テストを緩めて成功させる	期待値の根拠が曖昧	元の仕様と再現条件に戻る
同じ説明を繰り返す	参照情報が分散・古い	短い共通ルールを更新する

効果は、速さ・品質・負担を一緒に測る

導入前と同種の仕事を比べ、レビューと手戻りを含めた全体の変化を見る。

速さ

変更リードタイム

コミットから本番反映まで
レビュー待ち時間も記録

品質

不具合・手戻り

本番障害や差し戻しの件数
原因と修正工数を記録

負担

確認時間・総コスト

レビューと修正の時間
AI利用料と運用工数を記録

比較の条件：同じチーム・同種のタスク・同じ集計期間。

生成行数や採用率だけで成功とせず、品質の悪化とレビュー負荷も確認する。

4週間で、小さく試して継続を判断する

導入計画の例：1チーム・1種類のタスクから始める。期間と規模は調整する。



拡大は、品質・負担を許容範囲に保ち、チームで繰り返し実践できてから。

明日の1件は、このチェックから始める

開始時と完了時に確認する、持ち帰り用チェックリスト。

開始前

- ☐ 目的と対象外を説明できる
- ☐ 受け入れ条件が具体的
- ☐ 関連ファイルと実行方法がある
- ☐ データと実行権限を確認した

完了前

- ☐ 差分を読み、意図を説明できる
- ☐ テスト・解析の実行結果がある
- ☐ 未検証の範囲を明記した
- ☐ レビュー担当者が確認した

小さく任せる。結果を確かめる。学びをチームに残す。

参考資料・出典

2026年9月8日参照。一般化した実践手順と、独自の仕様例・導入案を含みます。

- | | |
|---|----------------------|
| [1] Anthropic Best practices for Claude Code
https://code.claude.com/docs/en/best-practices | タスクの具体化・文脈・実行による検証 |
| [2] GitHub Application card: GitHub Copilot Agents
https://docs.github.com/en/copilot/responsible-use/agents | 生成コードとAIレビューの限界・人の責任 |
| [3] OWASP LLM01:2025 Prompt Injection
https://genai.owasp.org/llmrisk/llm01-prompt-injection/ | 外部コンテンツを介した指示の混入 |
| [4] OWASP LLM06:2025 Excessive Agency
https://genai.owasp.org/llmrisk/llm062025-excessive-agency/ | 権限の最小化・重要操作の制御 |
| [5] DORA Software delivery performance metrics
https://dora.dev/guides/dora-metrics/ | 変更の流れと不安定性を測る観点 |
-

出典：デジタル庁デザインシステムウェブサイト <https://design.digital.go.jp/dads/> のコンテンツを加工して作成

本資料はデジタル庁が作成・監修したものではありません。